

PROJECT DOCUMENTATION · TECHNICAL REPORT

Legal Document Reader

A full-stack AI-powered legal-tech application that enables users to upload legal documents, perform semantic question answering, and generate structured legal document drafts — built with a RAG (Retrieval-Augmented Generation) pipeline backed by FAISS, FastEmbed, and Groq LLM.

CATEGORY

AI / Legal-Tech / Full-Stack

BACKEND

Python · FastAPI · FAISS · Groq

FRONTEND

React · Vite · Tailwind CSS · Redux

DATABASE

MongoDB Atlas · Motor (async)

DEPLOYMENT

Render (Backend) · Vercel (Frontend)

CONTENTS

1. Problem Statement
2. Project Overview
3. System Architecture
4. Tech Stack & Purpose
5. Backend — Functions & APIs

6. Frontend — Pages & Components
7. RAG Pipeline (Deep Dive)
8. Data Flow Diagrams
9. API Endpoint Reference
10. Deployment & Configuration

1. PROBLEM STATEMENT

Why This Application Exists

Legal documents — contracts, rental agreements, land registrations, lease agreements, and constitutional texts — are dense, lengthy, and written in complex legal language. The average person, including students, small business owners, and tenants, cannot easily understand these documents without professional legal counsel, which is expensive and inaccessible.

Traditional document reading tools provide no intelligence — they display text but cannot answer questions, extract key clauses, or summarize obligations. Users are left to manually search through dozens of pages to find specific information.

Core Problems

- Legal language is complex and difficult to interpret without expertise
- No affordable tool for ordinary users to query their own documents
- Scanned documents (images) are not machine-readable without OCR
- Generating a draft legal document requires a lawyer or expensive software
- General AI models hallucinate facts not present in the actual document

What This App Solves

- Upload any PDF or image of a legal document
- Ask plain-English questions and get answers grounded in the document
- Auto-summarize documents with Groq LLM
- Generate legal document drafts from templates
- Prevent AI hallucination via RAG — answers come from actual document text

Key insight: Unlike generic AI chatbots, this application uses Retrieval-Augmented Generation (RAG) to ensure answers are grounded in the uploaded document, not fabricated by the model. This is critical for legal contexts where accuracy is essential.

2. PROJECT OVERVIEW

What the Application Does

Feature 1

Document Upload & Processing

Upload PDF or image files. Text is extracted via PyMuPDF (text-based) or Tesseract OCR (scanned/image). The text is embedded and indexed in FAISS for fast similarity search.

Feature 2

Document Q&A Chat

Ask natural language questions about the uploaded document. The system retrieves the most relevant passage using vector similarity, then sends it to Groq LLM as context for a grounded answer.

Feature 3

Legal Document Generation

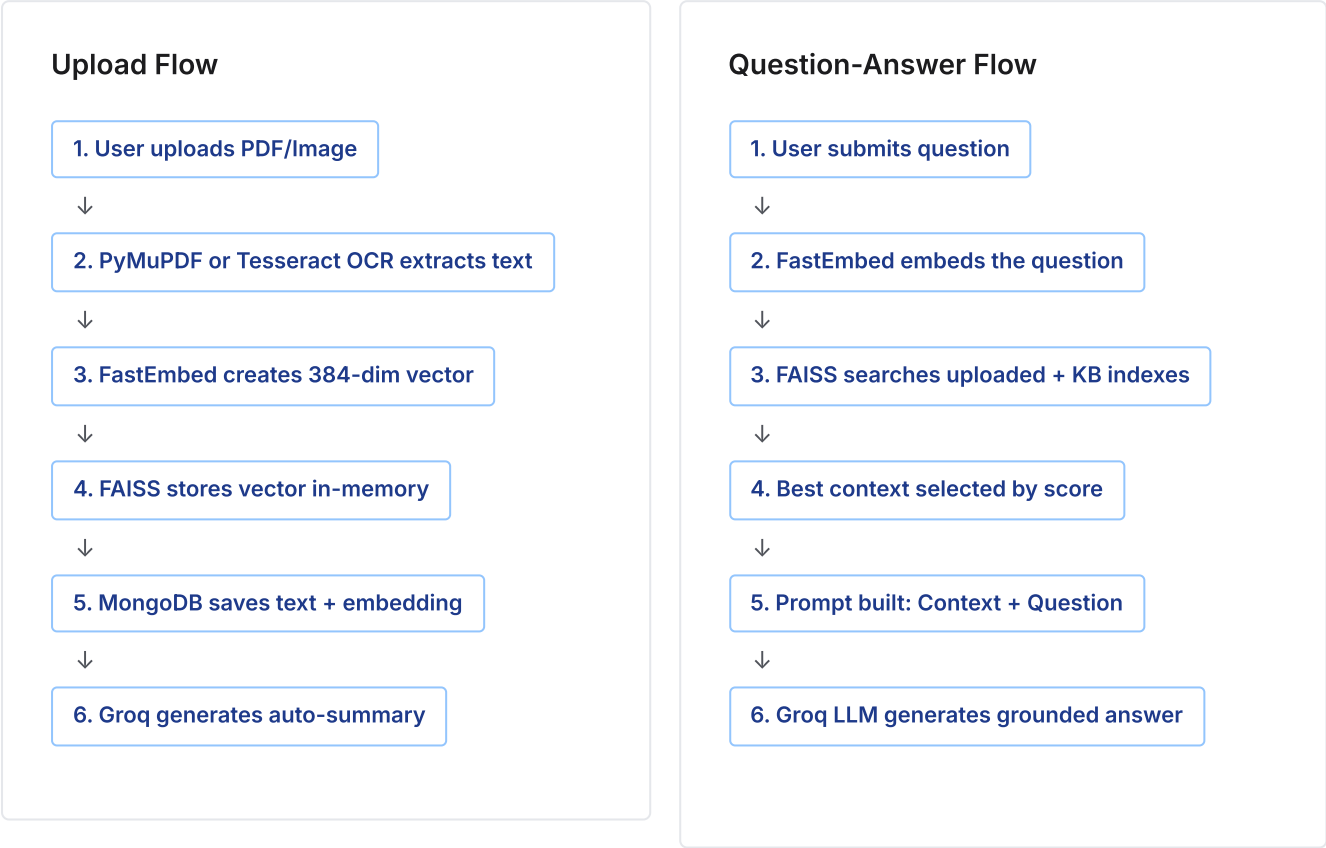
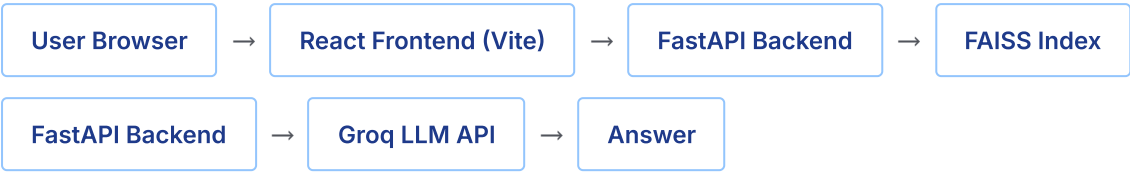
Select a document type (Rental Agreement, Lease, Land Registration, Building Registration). Provide the key details/clauses. The app generates a complete formatted legal document draft using a Groq-powered template.

Additional Capabilities

- **Knowledge Base:** A preloaded legal Q&A knowledge base (14,453 vectors) provides answers even without an uploaded document
- **Smart Context Selection:** The system automatically chooses between the uploaded document and the knowledge base based on similarity scores
- **Automatic Summarization:** Every uploaded document is summarized by Groq LLM immediately after processing
- **Persistence:** Documents and embeddings are stored in MongoDB Atlas so uploads persist across sessions
- **Authentication:** User signup/login with SHA-256 password hashing stored in MongoDB
- **Query Logging:** Every Q&A interaction is saved to MongoDB for audit/analysis

3. SYSTEM ARCHITECTURE

High-Level Architecture



4. TECH STACK & PURPOSE

Technology Choices Explained

Backend

Technology	Version	Purpose in This Project
Python	3.11	Primary backend language. Chosen for its rich AI/ML ecosystem (NumPy, FAISS, FastEmbed, Groq SDK)
FastAPI	0.115	Async REST API framework. Provides automatic OpenAPI docs, request validation via Pydantic, and high performance via async/await. All 5 endpoints (upload, chatbot, generate, auth) are FastAPI routes.
Uvicorn	0.34	ASGI server that runs FastAPI. Handles concurrent HTTP requests efficiently, critical for slow operations like OCR and LLM calls.
FAISS (faiss-cpu)	1.15	Facebook AI Similarity Search — an in-memory vector index. Used to store document embeddings and perform fast nearest-neighbour search when retrieving context for questions. Two FAISS IndexFlatL2 indexes: one for uploaded docs, one for the knowledge base.
FastEmbed	0.8	Lightweight embedding model library. Uses <code>sentence-transformers/all-MiniLM-L6-v2</code> to convert text into 384-dimensional float32 vectors. Used for both indexing documents and embedding user questions.
Groq SDK (groq)	latest	Async client for Groq's LLM API (OpenAI-compatible interface). Used for: (1) auto-summarizing uploaded documents, (2) generating answers to user questions with retrieved context, (3) generating legal document content. Model: <code>openai/gpt-oss-120b</code> with fallback options.
PyMuPDF (fitz)	1.26	PDF processing library. Extracts text from PDF pages. Falls back to Tesseract OCR if extracted text is too short (indicating a scanned/image-based PDF).
Pytesseract	0.3	Python wrapper for Google's Tesseract OCR engine. Used when PDFs contain scanned images (non-text) or when the uploaded file is a PNG/JPG image. Converts pixel images to machine-readable text.
Pillow (PIL)	11.2	Image processing library. Used alongside Pytesseract — renders PDF pages as images at 1.5x resolution for better OCR accuracy, and loads image files (PNG, JPG) for OCR processing.
Motor	3.7	Async MongoDB driver for Python. Used with <code>async/await</code> so database operations (document storage, user auth, query logging) don't block the FastAPI event loop.
MongoDB Atlas	cloud	Cloud NoSQL database. Three collections: <code>documents</code> (filename + extracted text + embedding), <code>users</code> (name + email + password_hash), <code>queries</code> (question + answer + source). Embeddings persist between server restarts.

Technology	Version	Purpose in This Project
Pydantic	2.11	Data validation for request/response models. Defines <code>ChatRequest</code> , <code>LegalDocumentRequest</code> , <code>SignupRequest</code> , <code>LoginRequest</code> — FastAPI automatically validates and serializes these.
python-dotenv	1.1	Loads environment variables from <code>.env</code> file. Used for <code>MONGO_URI</code> , <code>GROQ_API_KEY</code> , <code>ALLOWED_ORIGINS</code> — keeps secrets out of code.
NumPy	2.3	Numerical computing. Handles embedding vectors as float32 arrays, loads precomputed knowledge-base embeddings from <code>.npy</code> files, and manages FAISS vector operations.
hashlib (stdlib)	—	Python standard library. Used for SHA-256 password hashing before storing in MongoDB (no plaintext passwords stored).

Frontend

Technology	Version	Purpose in This Project
React	18.3	Core UI framework. All pages (Home, Document, Chatbot, Generate, Login, Signup) are React functional components using hooks (<code>useState</code> , <code>useRef</code> , <code>useEffect</code>).
Vite	6.2	Frontend build tool and dev server. Chosen over Create-React-App for its fast HMR (Hot Module Replacement) and optimized production builds. Configured with React plugin.
React Router DOM	7.2	Client-side routing. Defines 4 main routes: <code>/</code> (Home), <code>/upload-legal-doc</code> (Document), <code>/chatbot</code> (Chatbot), <code>/generate-legal-doc</code> (Generate). Uses <code>NavLink</code> for active-state navigation highlighting.
Redux Toolkit + React-Redux	2.6 / 9.2	Global state management. Stores the uploaded document's filename in a <code>filenameSlice</code> so it persists across page navigations (e.g., upload on Document page → accessible on Chatbot page without re-upload).
Axios	1.8	HTTP client for all API calls to FastAPI backend. Used in <code>Document.jsx</code> (<code>POST /upload-legal-doc/</code>), <code>Chatbot.jsx</code> (<code>POST /chatbot/</code>), and <code>Generate.jsx</code> (<code>POST /generate-legal-doc/</code>).
Tailwind CSS	3.4	Utility-first CSS framework. Used for all styling — layout (flexbox/grid), typography, colors, spacing, responsive breakpoints. Extended with custom design tokens: <code>navy</code> , <code>surface</code> , and <code>ink</code> color scales for a consistent professional palette.
React Dropzone	14.3	Drag-and-drop file input. Handles file selection, drag-active states, and accept filters (PDF/image). Used in the <code>FileDropzone</code> component on the upload page.
React Toastify	11.0	Toast notification system. Displays success/error notifications for auth actions (login/signup). Configured in <code>Layout.jsx</code> via <code>ToastContainer</code> .

Technology	Version	Purpose in This Project
React Icons	5.5	Icon library. Previously used for upload and send icons. Retained as a dependency.
clsx + tailwind-merge	2.1 / 3.0	Utility for conditionally merging Tailwind class names without conflicts. Used in the <code>cn()</code> helper in <code>lib/utils.jsx</code> , which is used by <code>FormButton</code> .
PostCSS + Autoprefixer	8.4 / 10.4	CSS processing pipeline. Processes Tailwind's <code>@tailwind</code> directives and adds vendor prefixes for cross-browser compatibility.

Infrastructure & Deployment

Technology	Purpose
Render.com	Hosts the FastAPI backend as a Docker Web Service. The Dockerfile installs all Python dependencies and starts uvicorn. Environment variables (GROQ_API_KEY, MONGO_URI, ALLOWED_ORIGINS) are configured in Render's dashboard.
Vercel	Hosts the React frontend. Root directory is set to <code>Frontend/</code> . Build command is <code>npm run build</code> . <code>VITE_API_URL</code> points to the Render backend URL. <code>vercel.json</code> rewrites all routes to <code>/index.html</code> for SPA routing.
Docker	Containerizes the backend for consistent deployment on Render. The Dockerfile installs system dependencies (tesseract-ocr) and Python packages from requirements.txt.
MongoDB Atlas	Cloud-hosted MongoDB cluster. Provides the SRV connection string used by Motor. Data persists independent of server restarts or redeployments.
Groq Cloud API	Remote LLM inference API. The backend sends prompts to Groq's servers and receives completions. Temperature is set to 0.1 for factual, deterministic responses. Model selection is dynamic with fallback logic.

Core Functions (app.py)

Utility

clean_text(text: str) → str

Normalizes extracted text by removing null bytes (\x00) and collapsing all whitespace (spaces, newlines, tabs) into a single space. Applied to all text output from PyMuPDF and Tesseract before any further processing.

Embedding

create_embedding(text: str) → np.ndarray

Lazily initializes the FastEmbed TextEmbedding model (loaded once on first call, cached globally). Embeds the input text using sentence-transformers/all-MiniLM-L6-v2 and returns a (1, 384) float32 NumPy array. This vector is used for both FAISS indexing and query search.

Knowledge Base

load_knowledge_base() → None

Called on application startup. Loads cleaned_legal_data.json (14,453 Q&A records) and the corresponding precomputed embeddings from legal_embeddings.npy . Adds all vectors to the kb_index FAISS index and stores formatted text in kb_store dict. Validates that JSON record count matches embedding count and dimensions are 384.

OCR / Text Extraction

extract_text_from_pdf_bytes(data: bytes) → str

Opens a PDF from raw bytes using PyMuPDF (fitz). Extracts text from all pages using page.get_text("text") . If the result is less than 30 characters (indicating a scanned/image-based PDF), falls back to OCR: renders each page at 1.5x magnification as a PNG, then runs pytesseract.image_to_string() . Returns cleaned, joined text.

OCR

extract_text_from_image_bytes(data: bytes) → str

Opens an image file (PNG/JPG) from bytes using Pillow and runs Tesseract OCR directly. Used when the uploaded file is an image rather than a PDF. Returns cleaned text.

RAG**add_uploaded_document(text: str, filename: str) → None**

Creates an embedding for the document text, then adds it to the `uploaded_index` FAISS index. Stores the filename and full text in `uploaded_store` dict keyed by the FAISS index position. Called after every successful document upload.

RAG**search_index(index, store, query: str, top_k=3) → list**

Embeds the query text, then calls `index.search()` to find the top-k nearest vectors using L2 distance. Converts L2 distance to a similarity score via $1.0 / (1.0 + \text{distance})$. Returns a list of `(similarity_score, text_or_dict)` tuples, skipping invalid indices.

RAG**retrieve_context(query: str, top_k=3) → (str, str)**

The core RAG retrieval function. Searches both `uploaded_index` and `kb_index` simultaneously. Uses smart selection logic: if the uploaded document's best similarity score is $\geq \max(0.25, \text{best_kb_score})$, prioritizes the uploaded document's context. Otherwise falls back to the knowledge base. Returns `(context_text, source)` where source is `"uploaded_document"`, `"knowledge_base"`, or `"none"`.

LLM**async get_groq_model() → str**

Queries the Groq API's model list once (cached after first call via `groq_model_checked` flag). Selects the first available model from a priority list: `gpt-oss-120b` → `gpt-oss-20b` → `qwen3.8-27b` → `allam-2-7b` → `groq/compound` → `groq/compound-mini`. Ensures the app works even if the configured model becomes unavailable.

LLM**async generate_groq(prompt: str, max_output_tokens=220) → str**

Calls the Groq Chat Completions API with a system prompt that instructs the model to act as a careful legal assistant and not invent facts. Sets `temperature=0.1` for deterministic output. Raises HTTP 502 if Groq returns an empty response. Returns the response text stripped of whitespace.

Summarization

async summarize_text(text: str) → str

Splits the document text into 1,800-character chunks (up to 12 chunks). Calls `generate_groq()` on each chunk with a summarization prompt and 160 max tokens. Joins all chunk summaries into a single result. Called automatically after every document upload.

6. API ENDPOINT REFERENCE

REST API — All Endpoints

GET /

Health/root endpoint. Returns `{"message": "Legal RAG API is running"}`.

GET /health

Detailed health check. Returns MongoDB connectivity status, number of KB vectors loaded (14,453), and number of uploaded document vectors in FAISS.

```
{ "status": "ok", "mongodb": true, "knowledge_base_vectors": 14453,
  "uploaded_vectors": 1 }
```

POST /auth/signup

Creates a new user account. Accepts `{"name", "email", "password"}`. Checks for duplicate email, hashes password with SHA-256, stores in MongoDB `users` collection.

POST /auth/login

Authenticates a user. Accepts `{"email", "password"}`. Hashes the password and queries MongoDB. Returns `{"success": true, "user": {"name", "email"}}` or HTTP 401.

POST /upload-legal-doc/

The most complex endpoint. Accepts a `multipart/form-data` file upload (PDF, PNG, JPG, JPEG). Pipeline:

- Validates file extension
- Extracts text (PyMuPDF → fallback OCR)
- Creates 384-dim embedding via FastEmbed
- Saves document (filename + text + embedding) to MongoDB
- Adds embedding to FAISS `uploaded_index`
- Auto-summarizes with Groq LLM

Returns: `{"message", "filename", "extracted_text" (first 500 chars), "summary"}`

POST /chatbot/

Answers a legal question using RAG. Accepts `{"question": "..."} .`

- Embeds the question via FastEmbed
- Searches uploaded document FAISS index
- Searches knowledge base FAISS index
- Selects best context source by similarity score
- Builds a grounded prompt and calls Groq LLM
- Saves Q&A pair + source to MongoDB `queries` collection

Returns: `{"answer": "...", "source": "uploaded_document|knowledge_base|none"}`

POST `/generate-legal-doc/`

Generates a legal document draft. Accepts `{"template": "rental agreement|land registration|...", "clauses": "..."} .` Matches template to a hardcoded template string, fills in the `{clauses}` placeholder with user-provided details, and returns the completed document text.

Returns: `{"legal_document": "... formatted document ..."} .`

7. FRONTEND — PAGES & COMPONENTS

React Component Structure

Page

Home.jsx — /

Dashboard-style landing page. Shows a hero section with the app's value proposition, two CTA buttons (Upload Document, Ask Questions), and a numbered "How it works" workflow section (Upload → Analyze → Ask → Get Answers). No fake stats or decorative elements.

Page

Document.jsx — /upload-legal-doc

Upload page with four UI states: **idle** (drop zone visible), **loading** (spinner + "Processing document..."), **success** (green banner → auto-redirects to chat in 800ms), **error** (red banner with specific error from backend). All states driven by a `status` state variable.

Page

Chatbot.jsx — /chatbot

Two-panel document analysis workspace. Left panel: document name, processing status, suggested questions (5 pre-filled), upload/remove actions. Right panel: full chat history with user/assistant distinction, three-dot loading indicator, empty state with guidance, keyboard shortcut hint (Enter to send). All connected to Redux for document filename.

Page

Generate.jsx — /generate-legal-doc

Two-column document generation workspace. Left column: document type select (4 options), purpose/clauses textarea, Generate button with loading spinner, Start Over button, Download HTML button. Right column: live document preview area with idle/loading/success/error states. Axios POST to `/generate-legal-doc/`.

Component

FileDropzone.jsx

Reusable drag-and-drop file input built on `react-dropzone`. Accepts PDF + images. Shows file info card (thumbnail for images, PDF badge, filename, file size in KB/MB, type label). Supports `isLoading` prop to disable interaction during upload. "Process Document" button disabled when no file is selected.

Component

Navbar.jsx

Sticky top navigation. Uses React Router's `NavLink` for automatic active-state highlighting (navy background on active route). Desktop: horizontal link row. Mobile: `<details>`-based hamburger dropdown menu. Brand name: "Legal Document Reader".

State Management — Redux

Store: `fileSlice` — holds a single `value: string` (the uploaded filename). This bridges the Document page (where the file is uploaded) and the Chatbot page (which reads the filename to show document info). The filename is dispatched via `setFilename(response.data.filename)` on upload success and cleared via `setFilename("")` on removal.

```
// fileSlice.jsx
const filenameSlice = createSlice({
  name: "filename",
  initialState: { value: "" },
  reducers: {
    setFilename: (state, action) => { state.value = action.payload }
  }
})
```

API Configuration — lib/api.js

Exports a single `API_URL` constant. Reads from `VITE_API_URL` environment variable with fallback to `http://127.0.0.1:8000` for local development. Trailing slashes are stripped. All Axios calls in the app reference this constant.

```
const API_URL = (import.meta.env.VITE_API_URL ||
"http://127.0.0.1:8000").replace(/\/$/, "");
export default API_URL;
```

How Retrieval-Augmented Generation Works Here

What is RAG? RAG is an AI pattern where, instead of asking an LLM to answer from memory, you first retrieve relevant text from a document store and inject it as context into the prompt. This prevents hallucination and grounds answers in real information.

Two-Source Retrieval Strategy

This app maintains two separate FAISS indexes simultaneously:

Index	Contents	Size	Source
kb_index	Preloaded legal Q&A knowledge base	14,453 vectors	cleaned_legal_data.json + legal_embeddings.npy
uploaded_index	User-uploaded documents	Dynamic (grows with uploads)	Computed live on upload, restored from MongoDB on startup

Context Selection Logic

The `retrieve_context()` function implements smart priority logic:

```
if uploaded_doc_score ≥ max(0.25, knowledge_base_score):
    → Use uploaded document context
elif knowledge_base has results:
    → Use knowledge base context
else:
    → No context (LLM answers from general knowledge)
```

The threshold of **0.25** ensures that an uploaded document is used even if slightly less similar than the KB, as long as it clears a minimum relevance bar. This prioritizes user-uploaded document context when it's reasonably relevant.

Prompt Engineering

System prompt (sent to Groq on every call):

```
"You are a careful legal document assistant. Answer clearly and accurately.
When context is provided, use only that context and do not invent facts."
```

Question prompt with context:

"Answer the legal question using ONLY the provided context.
If the context does not contain enough information, say that the document
does not provide enough information. Do not invent facts.

Context:
{retrieved_text[:10000]}

Question: {user_question}"

Similarity Score Formula

FAISS returns L2 (Euclidean) distances. The app converts these to similarity scores:

$$\text{similarity} = 1.0 / (1.0 + \text{l2_distance})$$

This gives scores between 0 and 1, where 1 = identical vectors (perfect match) and values approaching 0 = very dissimilar. The top-3 results are retrieved from each index for context assembly.

9. STARTUP & PERSISTENCE

Application Lifecycle

On Startup (`@app.on_event("startup")`)

- **load_knowledge_base():** Loads 14,453 legal Q&A records from JSON + precomputed embeddings into the in-memory `kb_index` FAISS index
- **MongoDB restore:** Queries all documents from `documents_collection` that have an `embedding` field, and re-adds them to `uploaded_index` . This means uploaded documents survive server restarts.

On Shutdown (`@app.on_event("shutdown")`)

Gracefully closes the MongoDB Motor client connection.

FAISS is in-memory only. FAISS indexes are not persisted to disk between restarts. However, since embeddings are stored in MongoDB, they are reconstructed from there on every startup. The knowledge base uses precomputed `.npy` files, so it loads instantly without re-embedding.

10. DEPLOYMENT & CONFIGURATION

Production Setup

Backend — Render.com

- Deployed as a **Docker Web Service**
- Start command: `uvicorn Backend.app:app --host 0.0.0.0 --port $PORT`
- Tesseract OCR must be installed in the Docker image
- Environment variables set in Render dashboard

Frontend — Vercel

- Root directory: `Frontend/`
- Build command: `npm run build`
- Output: `dist/`
- `VITE_API_URL` → Render backend URL
- `vercel.json` rewrites all routes to `index.html`

Environment Variables

Variable	Required	Purpose
MONGO_URI	Yes	MongoDB Atlas SRV connection string
MONGO_DB	No	Database name (default: <code>legal_assistant</code>)
GROQ_API_KEY	Yes	Groq Cloud API key for LLM access
GROQ_MODEL	No	Preferred LLM model (default: <code>openai/gpt-oss-120b</code>)
ALLOWED_ORIGINS	Yes (prod)	Comma-separated list of allowed CORS origins (e.g., Vercel URL)
VITE_API_URL	Yes (prod)	Frontend env var: points to backend base URL

Local Development — Quick Start

```
# Backend (from project root)
.\venv\Scripts\python.exe -m uvicorn app:app --reload --host 127.0.0.1 --port 8000

# Frontend (in a separate terminal)
cd Frontend
npm run dev
# → http://localhost:5173
```

Legal Document Reader — Project Technical Report

Generated: October 2026

Full-stack AI Legal-Tech Application · Python / FastAPI / React / FAISS / Groq